

2.5 Casting and Generics

In this section, we discuss casting among reference variables, as well as a technique, called generics, that allows us to define methods and classes that work with a variety of data types without the need for explicit casting.

2.5.1 Casting

We begin our discussion with methods for type conversions for objects.

Widening Conversions

A **widening conversion** occurs when a type T is converted into a “wider” type U . The following are common cases of widening conversions:

- T and U are class types and U is a superclass of T .
- T and U are interface types and U is a superinterface of T .
- T is a class that implements interface U .

Widening conversions are automatically performed to store the result of an expression into a variable, without the need for an explicit cast. Thus, we can directly assign the result of an expression of type T into a variable v of type U when the conversion from T to U is a widening conversion. When discussing polymorphism on page 68, we gave the following example of an implicit widening cast, assigning an instance of the narrower `PredatoryCreditCard` class to a variable of the wider `CreditCard` type:

```
CreditCard card = new PredatoryCreditCard(...); // parameters omitted
```

The correctness of a widening conversion can be checked by the compiler and its validity does not require testing by the Java runtime environment during program execution.

Narrowing Conversions

A **narrowing conversion** occurs when a type T is converted into a “narrower” type S . The following are common cases of narrowing conversions:

- T and S are class types and S is a subclass of T .
- T and S are interface types and S is a subinterface of T .
- T is an interface implemented by class S .

In general, a narrowing conversion of reference types requires an explicit cast. Also, the correctness of a narrowing conversion may not be verifiable by the compiler. Thus, its validity should be tested by the Java runtime environment during program execution.

The example code fragment below shows how to use a cast to perform a narrowing conversion from type `PredatoryCreditCard` to type `CreditCard`.

```
CreditCard card = new PredatoryCreditCard(...);    // widening
PredatoryCreditCard pc = (PredatoryCreditCard) card; // narrowing
```

Although variable `card` happens to reference an instance of a `PredatoryCreditCard`, the variable has declared type, `CreditCard`. Therefore, the assignment `pc = card` is a narrowing conversion and requires an explicit cast that will be evaluated at runtime (as not all cards are predatory).

Casting Exceptions

In Java, we can cast an object reference *o* of type *T* into a type *S*, provided the object *o* is referring to is actually of type *S*. If, on the other hand, object *o* is not also of type *S*, then attempting to cast *o* to type *S* will throw an exception called `ClassCastException`. We illustrate this rule in the following code fragment, using Java's `Number` abstract class, which is a superclass of both `Integer` and `Double`.

```
Number n;
Integer i;
n = new Integer(3);
i = (Integer) n;           // This is legal
n = new Double(3.1415);
i = (Integer) n;           // This is illegal
```

To avoid problems such as this and to avoid peppering our code with try-catch blocks every time we perform a cast, Java provides a way to make sure an object cast will be correct. Namely, it provides an operator, **instanceof**, that allows us to test whether an object variable is referring to an object that belongs to a particular type. The syntax for this operator is *objectReference instanceof referenceType*, where *objectReference* is an expression that evaluates to an object reference and *referenceType* is the name of some existing class, interface, or enum (Section 1.3). If *objectReference* is indeed an instance satisfying *referenceType*, then the operator returns **true**; otherwise, it returns **false**. Thus, we can avoid a `ClassCastException` from being thrown in the code fragment above by modifying it as follows:

```
Number n;
Integer i;
n = new Integer(3);
if (n instanceof Integer)
    i = (Integer) n;           // This is legal
n = new Double(3.1415);
if (n instanceof Integer)
    i = (Integer) n;           // This will not be attempted
```

Casting with Interfaces

Interfaces allow us to enforce that objects implement certain methods, but using interface variables with concrete objects sometimes requires casting. Suppose we declare a `Person` interface as shown in Code Fragment 2.14. Note that method `equals` of the `Person` interface takes one parameter of type `Person`. Thus, we can pass an object of any class implementing the `Person` interface to this method.

```

1 public interface Person {
2     public boolean equals(Person other);           // is this the same person?
3     public String getName();                       // get this person's name
4     public int getAge();                           // get this person's age
5 }
```

Code Fragment 2.14: Interface `Person`.

In Code Fragment 2.15, we show a class, `Student`, that implements `Person`. Because the parameter to `equals` is a `Person`, the implementation must not assume that it is necessarily of type `Student`. Instead, it first uses the **`instanceof`** operator at line 15, returning **`false`** if the argument is not a student (since it surely is not the student in question). Only after verifying that the parameter is a student, is it explicitly cast to a `Student`, at which point its `id` field can be accessed.

```

1 public class Student implements Person {
2     String id;
3     String name;
4     int age;
5     public Student(String i, String n, int a) {           // simple constructor
6         id = i;
7         name = n;
8         age = a;
9     }
10    protected int studyHours() { return age/2;}           // just a guess
11    public String getID() { return id;}                   // ID of the student
12    public String getName() { return name;}               // from Person interface
13    public int getAge() { return age;}                    // from Person interface
14    public boolean equals(Person other) {                 // from Person interface
15        if (!(other instanceof Student)) return false;   // cannot possibly be equal
16        Student s = (Student) other;                    // explicit cast now safe
17        return id.equals(s.id);                          // compare IDs
18    }
19    public String toString() {                             // for printing
20        return "Student(ID:" + id + ", Name:" + name + ", Age:" + age + ")";
21    }
22 }
```

Code Fragment 2.15: Class `Student` implementing interface `Person`.

2.5.2 Generics

Java includes support for writing *generic* classes and methods that can operate on a variety of data types while often avoiding the need for explicit casts. The generics framework allows us to define a class in terms of a set of *formal type parameters*, which can then be used as the declared type for variables, parameters, and return values within the class definition. Those formal type parameters are later specified when using the generic class as a type elsewhere in a program.

To better motivate the use of generics, we consider a simple case study. Often, we wish to treat a pair of related values as a single object, for example, so that the pair can be returned from a method. A solution is to define a new class whose instances store both values. This is our first example of an object-oriented design pattern known as the *composition design pattern*. If we know, for example, that we want a pair to store a string and a floating-point number, perhaps to store a stock ticker label and a price, we could easily design a custom class for that purpose. However, for another purpose, we might want to store a pair that consists of a Book object and an integer that represents a quantity. The goal of generic programming is to be able to write a single class that can represent all such pairs.

The generics framework was not a part of the original Java language; it was added as part of Java SE 5. Prior to that, generic programming was implemented by relying heavily on Java's Object class, which is the universal supertype of all objects (including the wrapper types corresponding to primitives). In that "classic" style, a generic pair might be implemented as shown in Code Fragment 2.16.

```
1 public class ObjectPair {  
2     Object first;  
3     Object second;  
4     public ObjectPair(Object a, Object b) {           // constructor  
5         first = a;  
6         second = b;  
7     }  
8     public Object getFirst() { return first; }  
9     public Object getSecond() { return second; }  
10 }
```

Code Fragment 2.16: Representing a generic pair of objects using a classic style.

An ObjectPair instance stores the two objects that are sent to the constructor, and provides individual accessors for each component of the pair. With this definition, a pair can be declared and instantiated with the following command:

```
ObjectPair bid = new ObjectPair("ORCL", 32.07);
```

This instantiation is legal because the parameters to the constructor undergo widening conversions. The first parameter, "ORCL", is a String, and thus also an Object.

The second parameter is a **double**, but it is automatically boxed into a `Double`, which then qualifies as an `Object`. (For the record, this is not quite the “classic” style, as automatic boxing was not introduced until Java SE 5.)

The drawback of the classic approach involves use of the accessors, both of which formally return an `Object` reference. Even if we know that the first object is a string in our application, we cannot legally make the following assignment:

```
String stock = bid.getFirst();           // illegal; compile error
```

This represents a narrowing conversion from the declared return type of `Object` to the variable of type `String`. Instead, an explicit cast is required, as follows:

```
String stock = (String) bid.getFirst(); // narrowing cast: Object to String
```

With the classic style for generics, code became rampant with such explicit casts.

Using Java's Generics Framework

With Java's generics framework, we can implement a pair class using formal type parameters to represent the two relevant types in our composition. An implementation using this framework is given in Code Fragment 2.17.

```

1 public class Pair<A,B> {
2     A first;
3     B second;
4     public Pair(A a, B b) {           // constructor
5         first = a;
6         second = b;
7     }
8     public A getFirst() { return first; }
9     public B getSecond() { return second; }
10 }
```

Code Fragment 2.17: Representing a pair of objects with generic type parameters.

Angle brackets are used at line 1 to enclose the sequence of formal type parameters. Although any valid identifier can be used for a formal type parameter, single-letter uppercase names are conventionally used (in this example, `A` and `B`). We may then use these type parameters within the body of the class definition. For example, we declare instance variable, `first`, to have type `A`; we similarly use `A` as the declared type for the first constructor parameter and for the return type of method, `getFirst`.

When subsequently declaring a variable with such a parameterize type, we must explicitly specify *actual type parameters* that will take the place of the generic formal type parameters. For example, to declare a variable that is a pair holding a stock-ticker string and a price, we write the following:

```
Pair<String,Double> bid;
```

Effectively, we have stated that we wish to have `String` serve in place of type `A`, and `Double` serve in place of type `B` for the pair known as `bid`. The actual types for generic programming must be object types, which is why we use the wrapper class `Double` instead of the primitive type **`double`**. (Fortunately, the automatic boxing and unboxing will work in our favor.)

We can subsequently instantiate the generic class using the following syntax:

```
bid = new Pair<>("ORCL", 32.07);           // rely on type inference
```

After the **`new`** operator, we provide the name of the generic class, then an empty set of angle brackets (known as the “diamond”), and finally the parameters to the constructor. An instance of the generic class is created, with the actual types for the formal type parameters determined based upon the original declaration of the variable to which it is assigned (`bid` in this example). This process is known as *type inference*, and was introduced to the generics framework in Java SE 7.

It is also possible to use a style that existed prior to Java SE 7, in which the generic type parameters are explicitly specified between angle brackets during instantiation. Using that style, our previous example would be implemented as:

```
bid = new Pair<String,Double>("ORCL", 32.07); // give explicit types
```

However, it is important that one of the two above styles be used. If angle brackets are entirely omitted, as in the following example,

```
bid = new Pair("ORCL", 32.07);           // classic style
```

this reverts to the classic style, with `Object` automatically used for all generic type parameters, and resulting in a compiler warning when assigning to a variable with more specific types.

Although the syntax for the declaration and instantiation of objects using the generics framework is slightly more cluttered than the classic style, the advantage is that there is no longer any need for explicit narrowing casts from `Object` to a more specific type. Continuing with our example, since `bid` was declared with actual type parameters `<String,Double>`, the return type of the `getFirst()` method is `String`, and the return type of the `getSecond()` method is `Double`. Unlike the classic style, we can make the following assignments without any explicit casting (although there is still an automatic unboxing of the `Double`):

```
String stock = bid.getFirst();  
double price = bid.getSecond();
```

Generics and Arrays

There is an important caveat related to generic types and the use of arrays. Although Java allows the declaration of an array storing a parameterized type, it does not technically allow the instantiation of new arrays involving those types. Fortunately, it allows an array defined with a parameterized type to be initialized with a newly created, nonparametric array, which can then be cast to the parameterized type. Even so, this latter mechanism causes the Java compiler to issue a warning, because it is not 100% type-safe.

We will see this issue arise in two ways:

- Code outside a generic class may wish to declare an array storing instances of the generic class with actual type parameters.
- A generic class may wish to declare an array storing objects that belong to one of the formal parameter types.

As an example of the first use case, we continue with our stock market example and presume that we would like to keep an array of `Pair<String,Double>` objects. Such an array can be declared with a parameterized type, but it must be instantiated with an *unparameterized* type and then cast back to the parameterized type. We demonstrate this usage in the following:

```
Pair<String,Double>[ ] holdings;
holdings = new Pair<String,Double>[25];    // illegal; compile error
holdings = new Pair[25];                  // correct, but warning about unchecked cast
holdings[0] = new Pair<>("ORCL", 32.07);  // valid element assignment
```

As an example of the second use case, assume that we want to create a generic `Portfolio` class that can store a fixed number of generic entries in an array. If the class uses `<T>` as a parameterized type, it can declare an array of type `T[ ]`, but it cannot directly instantiate such an array. Instead, a common approach is to instantiate an array of type `Object[ ]`, and then make a narrowing cast to type `T[ ]`, as shown in the following:

```
public class Portfolio<T> {
    T[ ] data;
    public Portfolio(int capacity) {
        data = new T[capacity];           // illegal; compiler error
        data = (T[ ]) new Object[capacity]; // legal, but compiler warning
    }
    public T get(int index) { return data[index]; }
    public void set(int index, T element) { data[index] = element; }
}
```

Generic Methods

The generics framework allows us to define generic versions of individual methods (as opposed to generic versions of entire classes). To do so, we include a generic formal type declaration among the method modifiers.

For example, we show below a nonparametric GenericDemo class with a parameterized static method that can reverse an array containing elements of any object type.

```
public class GenericDemo {  
    public static <T> void reverse(T[] data) {  
        int low = 0, high = data.length - 1;  
        while (low < high) {           // swap data[low] and data[high]  
            T temp = data[low];  
            data[low++] = data[high];   // post-increment of low  
            data[high--] = temp;       // post-decrement of high  
        }  
    }  
}
```

Note the use of the <T> modifier to declare the method to be generic, and the use of the type T within the method body, when declaring the local variable, temp.

The method can be called using the syntax, GenericDemo.reverse(books), with type inference determining the generic type, assuming books is an array of some object type. (This generic method cannot be applied to primitive arrays, because autoboxing does not apply to entire arrays.)

As an aside, we note that we could have implemented a reverse method equally well using a classic style, acting upon an Object[] array.

Bounded Generic Types

By default, when using a type name such as T in a generic class or method, a user can specify any object type as the actual type of the generic. A formal parameter type can be restricted by using the **extends** keyword followed by a class or interface. In that case, only a type that satisfies the stated condition is allowed to substitute for the parameter. The advantage of such a bounded type is that it becomes possible to call any methods that are guaranteed by the stated bound.

As an example, we might declare a generic ShoppingCart that could only be instantiated with a type that satisfied the Sellable interface (from Code Fragment 2.8 on page 77). Such a class would be declared beginning with the line:

```
public class ShoppingCart<T extends Sellable> {
```

Within that class definition, we would then be allowed to call methods such as description() and lowestPrice() on any instances of type T.